

Padrões de Software no Sistema de Gestão de Clínicas

Índice

1	Introdução	2
2	Padrões GoF Adotados	2
2.1	1. Singleton	2
2.2	2. Adapter	3
2.3	3. Decorator	4
2.4	4. Proxy	4
2.5	5. Chain of Responsibility	5
2.6	6. Observer	6
2.7	7. State	6
2.8	8. Template Method	7
3	Conclusão	8

1 Introdução

Este relatório apresenta uma análise aprofundada dos padrões de projeto clássicos do catálogo **GoF (Gang of Four)** identificados e aplicados na base de código do sistema de gestão de clínicas médicas. A aplicação é baseada em uma arquitetura de microsserviços em Python, utilizando os frameworks FastAPI e SQLAlchemy.

O objetivo do documento é descrever os padrões de projeto que foram efetivamente utilizados e porquê. Mapeando e justificando o reuso de soluções estruturais clássicas do catálogo de 23 padrões originais do GoF, o relatório fundamenta o design com trechos reais do código-fonte e diagramas de classe em vetor nativo LaTeX (TikZ).

De acordo com as restrições da atividade, o foco deste relatório é puramente direcionado às implementações de padrões de projeto orientados a objetos clássicos que auxiliam a execução e o desacoplamento das funcionalidades internas do sistema.

2 Padrões GoF Adotados

Abaixo são detalhados os 8 padrões GoF identificados e aplicados no código-fonte dos microsserviços.

2.1 1. Singleton

Definição: O padrão Singleton garante que uma classe tenha apenas uma única instância em todo o ciclo de vida da aplicação, fornecendo um ponto de acesso global a essa instância.

Justificativa no Projeto: No desenvolvimento de microsserviços que interagem com bancos de dados relacionais, a criação repetida de pools de conexões e engines de mapeamento ORM gera overhead severo e pode esgotar o limite de conexões do SGBD. Portanto, em cada um dos microsserviços, a inicialização do motor de conexão (**engine**) e da fábrica de sessões (**SessionLocal**) do SQLAlchemy é feita uma única vez por processo. Em Python, a importação de módulos atua nativamente como um Singleton, pois o interpretador carrega o arquivo apenas uma vez e armazena sua referência no cache de módulos (**sys.modules**), distribuindo a mesma instância em todas as importações subsequentes. A instância global do aplicativo FastAPI (**app**) também funciona como um Singleton que coordena as rotas HTTP e middlewares.

Evidência no Código: O trecho abaixo em `ms-cadastros/database.py` demonstra a criação das instâncias únicas compartilhadas por todo o microsserviço:

```
1 from sqlalchemy import create_engine
2 from sqlalchemy.orm import declarative_base
3 from sqlalchemy.orm import sessionmaker
4 import os
5
6 SQLALCHEMY_DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./test.db")
7
8 # Instância única do motor de banco de dados (Singleton gerenciado pelo
   interpretador)
```

```

9 engine = create_engine(SQLALCHEMY_DATABASE_URL)
10 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=
    engine)
11
12 Base = declarative_base()

```

Listing 1: Instanciação única do Engine e SessionLocal (ms-cadastros/database.py)

2.2 2. Adapter

Definição: O padrão Adapter converte a interface de uma classe em outra interface esperada pelo cliente. O padrão permite que classes com interfaces incompatíveis trabalhem juntas.

Justificativa no Projeto: No sistema, as tabelas de banco de dados mapeadas por entidades ORM (SQLAlchemy) possuem tipos e estruturas rígidos que não correspondem diretamente ao formato JSON trafegado na rede. Para resolver essa incompatibilidade, são utilizados Schemas Pydantic como adaptadores (ou Data Transfer Objects - DTOs). O Pydantic adapta dados estruturados brutos (como dicionários Python e objetos ORM com atributos lazy) em saídas estruturadas JSON validadas automaticamente, utilizando a flag interna `from_attributes = True`. O próprio SQLAlchemy atua como um adaptador de dialetos de banco de dados, permitindo que o código em Python funcione sem alterações tanto com SQLite quanto com PostgreSQL.

Evidência no Código: Em ms-cadastros/schemas.py, a classe de configuração habilita a adaptação direta das entidades ORM para respostas HTTP JSON:

```

1 from pydantic import BaseModel
2
3 class PacienteBase(BaseModel):
4     nome: str
5     cpf: str
6     email: str
7     telefone: str
8
9 class Paciente(PacienteBase):
10     id: int
11
12     class Config:
13         # Configuração que adapta modelos ORM (SQLAlchemy) para DTOs
14         Pydantic
15         from_attributes = True

```

Listing 2: Esquema de adaptação de dados ORM para JSON (ms-cadastros/schemas.py)

No manipulador de requisições de ms-cadastros/main.py, o adaptador converte os dados enviados no request em objetos de banco de dados:

```

28 @app.post("/pacientes", response_model=schemas.Paciente)
29 def create_paciente(paciente: schemas.PacienteCreate, db: Session =
    Depends(get_db)):
30     # Adapta o payload Pydantic para o dicionário esperado pelo modelo
    SQLAlchemy
31     db_paciente = models.Paciente(**paciente.model_dump())
32     db.add(db_paciente)

```

```

33 db.commit()
34 db.refresh(db_paciente)
35 return db_paciente

```

Listing 3: Uso do Adapter na criação de entidades (ms-cadastros/main.py)

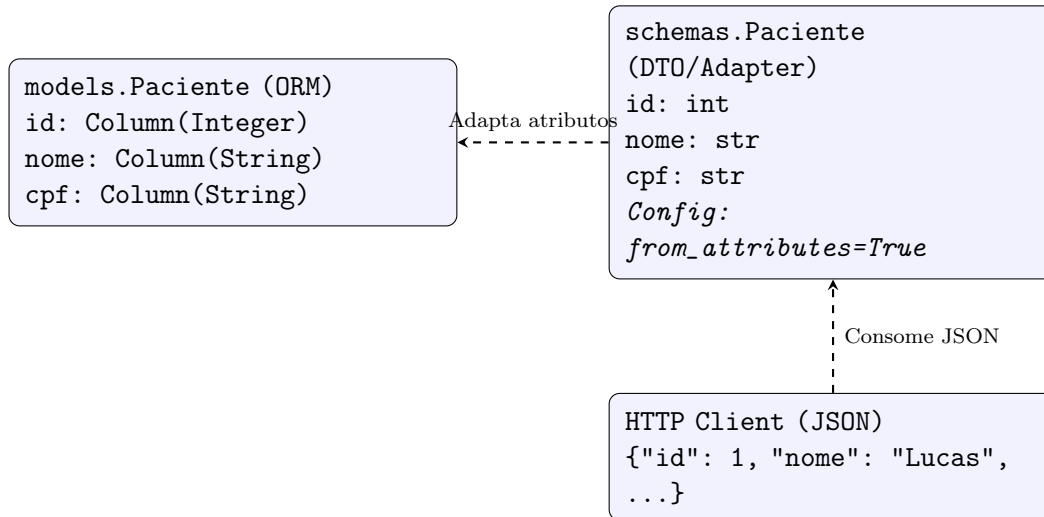


Figura 1: Diagrama do Padrão Adapter (Adaptação ORM → JSON DTO)

2.3 3. Decorator

Definição: O padrão Decorator anexa responsabilidades adicionais a um objeto dinamicamente, fornecendo uma alternativa flexível ao uso de subclasses para estender funcionalidade.

Justificativa no Projeto: No desenvolvimento com FastAPI, os Decorators nativos do Python (@) são utilizados para encapsular as funções que contêm a lógica de negócio dos endpoints. Em vez de acoplar as rotas e verbos HTTP dentro da lógica das funções, os decoradores interceptam as requisições recebidas do servidor web ASGI e injetam responsabilidades como roteamento URL, serialização automática, injeção de parâmetros e tratadores de eventos de ciclo de vida de forma declarativa e dinâmica.

Evidência no Código: Em ms-agendamentos/main.py, as rotas e validações são decoradas na inicialização:

```

87 @app.post("/consultas", response_model=schemas.Consulta)
88 def create_consulta(consulta: schemas.ConsultaCreate, db: Session =
    Depends(get_db)):
89     # ... Lógica interna do endpoint

```

Listing 4: Decoração de rota HTTP e metadados no FastAPI (ms-agendamentos/main.py)

2.4 4. Proxy

Definição: O padrão Proxy fornece um substituto ou marcador de posição para outro objeto para controlar o acesso a ele.

Justificativa no Projeto: Três variações do padrão Proxy são identificadas na base de código:

1. **Virtual Proxy (SQLAlchemy Lazy Loading):** Ao consultar um objeto de banco de dados, o ORM gera proxies que interceptam o acesso a propriedades de relacionamento. O banco de dados só é consultado quando o atributo é explicitamente acessado no código Python.
2. **Smart Reference / Intercepting Proxy (Injeção de Dependências):** O construtor `Depends` do FastAPI atua como um proxy de interceptação. Ele gerencia o ciclo de vida da conexão do banco de dados (`get_db`) e o controle de acesso por roles administrativas (`check_admin`), impedindo que a execução atinja a função interna caso as pré-condições falhem.
3. **Remote Proxy (Integração de Rede):** A função cliente de chamada síncrona HTTP baseada em `httpx.get` serve como um Remote Proxy para o microserviço de cadastros, encapsulando as requisições de rede físicas sob uma assinatura de método Python simples.

Evidência no Código: Em `ms-agendamentos/main.py`, a validação de acesso administrativo e a chamada remota demonstram os seus comportamentos de Proxy:

```
120 @app.post("/consultas/{consulta_id}/aprovar-cancelamento", dependencies
      =[Depends(check_admin)])
121 def aprovar_cancelamento(consulta_id: int, db: Session = Depends(get_db)
      ):
122     # ...
```

Listing 5: Proxy de Autenticação (`check_admin`) e Dependência de Sessão DB (`ms-agendamentos/main.py`)

Abaixo, a chamada remota que funciona como Remote Proxy encapsulando a integração de rede:

```
89     # 1. Validar Paciente via requisição remota ao MS-Cadastros
90     try:
91         resp_paciente = httpx.get(f"{MS_CADASTROS_URL}/interna/validar-
paciente/{consulta.paciente_id}", timeout=5.0)
92         if resp_paciente.status_code != 200:
93             raise HTTPException(status_code=400, detail="Paciente invá
lido ou não encontrado.")
94     except httpx.RequestError:
95         raise HTTPException(status_code=503, detail="Serviço de
Cadastros indisponível.")
```

Listing 6: Chamada remota de validação encapsulada (`ms-agendamentos/main.py`)

2.5 5. Chain of Responsibility

Definição: O padrão Chain of Responsibility evita o acoplamento do remetente de uma solicitação ao seu receptor, ao dar a mais de um objeto a oportunidade de tratar a solicitação. Encadeia os objetos receptores e passa a solicitação ao longo da cadeia até que um objeto a trate.

Justificativa no Projeto: O processamento de requisições HTTP em um servidor web como o FastAPI é estruturado em torno do conceito de middlewares. O `CORSMiddleware` e outros tratadores globais são registrados na aplicação formando uma lista sequencial de tratamento de requisições. Cada middleware representa um elo da cadeia: recebe a requisição, executa validações ou modificações de cabeçalhos (como políticas de origens de CORS) e decide se passa a requisição adiante na cadeia para o próximo middleware (ou manipulador final de rota) ou se interrompe o fluxo retornando uma resposta de erro imediata.

Evidência no Código: Instanciação e registro da cadeia de tratamento CORS em `ms-cadastros/main.py`:

```
14 app.add_middleware(  
15     CORSMiddleware,  
16     allow_origins=["*"],  
17     allow_credentials=True,  
18     allow_methods=["*"],  
19     allow_headers=["*"],  
20 )
```

Listing 7: Configuração de middleware na cadeia de requisições (`ms-cadastros/main.py`)

2.6 6. Observer

Definição: O padrão Observer define uma dependência um-para-muitos entre objetos, de modo que quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente.

Justificativa no Projeto: O ciclo de vida do servidor web FastAPI atua como o sujeito observado (Subject). O microserviço de faturamento precisa iniciar um worker em segundo plano de sincronização de cobranças pendentes assim que a infraestrutura do servidor estiver totalmente pronta e ativa. Registra-se uma função observadora no evento de inicialização ("`startup`") do aplicativo. Quando o loop de eventos detecta a conclusão do bootstrap do servidor, o gatilho é disparado, notificando o observador correspondente que inicia a tarefa assíncrona.

Evidência no Código: Trecho de `ms-faturamento/main.py` registrando o escutador de evento de inicialização:

```
85 @app.on_event("startup")  
86 async def startup_event():  
87     # Dispara concorrentemente o worker assíncrono ao ser notificado do  
88     startup  
89     asyncio.create_task(sync_worker())
```

Listing 8: Registro de observador para evento de startup do servidor (`ms-faturamento/main.py`)

2.7 7. State

Definição: O padrão State permite que um objeto altere seu comportamento quando seu estado interno muda. O objeto parecerá ter mudado de classe.

Justificativa no Projeto: As consultas médicas possuem estados bem definidos em seu ciclo de vida: PENDENTE, CONFIRMADA, CANCELADA e CONCLUÍDA. Cada um desses estados permite ou bloqueia determinadas ações de negócios. Por exemplo, uma consulta que já está concluída ou cancelada não pode ser remarcada ou cancelada novamente. O comportamento dos endpoints do MS-Agendamentos muda dinamicamente de acordo com o estado do registro persistido na tabela do banco de dados. Embora no projeto não tenham sido criadas classes separadas para representar cada estado (como exige o padrão clássico OO do GoF, o que geraria complexidade desnecessária para uma aplicação Python), a modelagem utiliza uma abordagem adaptada a microsserviços REST, onde o estado é persistido na coluna `status` e validado ativamente pelos handlers de rotas.

Evidência no Código: Em `ms-agendamentos/main.py`, o manipulador verifica o estado atual para decidir se a operação de remarcação é válida:

```
140 @app.put("/consultas/{consulta_id}/remarcar")
141 def remarcar_consulta(consulta_id: int, payload: schemas.
    RemarcarConsulta, db: Session = Depends(get_db)):
142     consulta = db.query(models.Consulta).filter(models.Consulta.id ==
        consulta_id).first()
143     if not consulta:
144         raise HTTPException(status_code=404, detail="Consulta não
            encontrada")
145
146     # O comportamento da transição varia conforme o estado atual
147     if consulta.status in ["CONCLUÍDA", "CANCELADA"]:
148         raise HTTPException(status_code=400, detail="Não é possível
            remarcar uma consulta concluída ou cancelada.")
149
150     verificar_escala_e_conflito(consulta.medico_id, payload.data_hora,
        db, consulta_id_excluir=consulta_id)
151
152     consulta.data_hora = payload.data_hora
153     consulta.status = "CONFIRMADA"
154     db.commit()
155     return {"message": "Consulta remarcada com sucesso.", "nova_data":
        consulta.data_hora}
```

Listing 9: Transições e validações de estado da consulta (`ms-agendamentos/main.py`)

2.8 8. Template Method

Definição: O padrão Template Method define o esqueleto de um algoritmo em uma operação, delegando alguns passos para subclasses. Permite que subclasses redefinam certos passos de um algoritmo sem alterar a estrutura do próprio algoritmo.

Justificativa no Projeto: Os modelos ORM SQLAlchemy herdando da classe declarativa `Base` e os schemas Pydantic herdando de `BaseModel` implementam a essência do padrão Template Method. As superclasses implementam todos os algoritmos fundamentais de validação de dados, mapeamento de metadados, escuta de eventos internos e sincronização SQL. As subclasses (como `Paciente`, `Medico`, `Consulta` e seus schemas correspondentes) fornecem apenas as definições concretas das propriedades (como nomes de colunas, restrições exclusivas e tipos de dados), preenchendo as lacunas do esqueleto de funcionamento fornecido pelas bibliotecas.

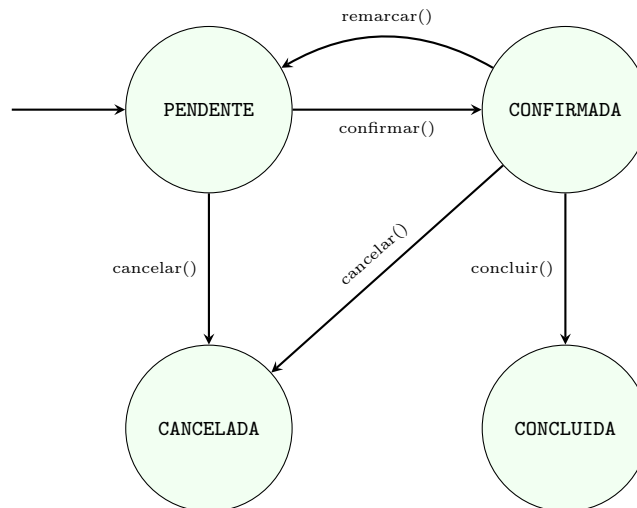


Figura 2: Diagrama de Transição de Estados da Consulta

Evidência no Código: Modelo de dados no arquivo `models.py` do agendamento:

```

5 class Consulta(Base):
6     # Sobrescreve as variáveis do template da classe pai Base do
    SQLAlchemy
7     __tablename__ = "consultas"
8
9     id = Column(Integer, primary_key=True, index=True)
10    paciente_id = Column(Integer, index=True)
11    medico_id = Column(Integer, index=True)
12    data_hora = Column(DateTime)
13    status = Column(String, default="PENDENTE")

```

Listing 10: Implementação concreta preenchendo o esqueleto da superclasse (ms-agendamentos/models.py)

3 Conclusão

A análise técnica do código-fonte do sistema de gestão de clínicas permitiu mapear claramente o uso pragmático dos padrões GoF em uma base de código baseada em microsserviços.

Constatou-se que o uso de frameworks consolidados como FastAPI e SQLAlchemy abstrai muitos dos padrões estruturais clássicos, como **Singleton**, **Adapter**, **Proxy** e **Template Method**, fornecendo-os nativamente à infraestrutura de roteamento e mapeamento relacional. Adicionalmente, padrões comportamentais como **State** e **Observer** foram implementados em conformidade com as restrições arquiteturais REST, controlando transições de ciclo de vida e monitorando eventos de ciclo de vida de containers.

A exclusão deliberada dos demais 15 padrões de projeto clássicos demonstra decisões de engenharia pragmáticas focadas no reuso de recursos nativos e no controle de complexidade do ecossistema de software.